专升本 **Java** 程序设计模拟卷-卷 **4**

fifine-专升本

2026 年 3 月 2 日

fifine专升本

1 单项选择题（共 20 小题，每小题 2 分，共 40 分）

1. 下列哪项不属于 Java 语言的关键字？

A. goto

B. const

C. null

D. native

答案与深度解析 答案：**C. null**

【核心认知框架】

- 题目本质：考查 Java 关键字保留字与字面量的区分，特别是保留但未使用的关键字。
- 关键区分点：`goto`/`const` 是保留关键字但禁用，`null` 是字面量而非关键字。
- 命题人意图：测试考生对 JLS（Java Language Specification）关键字列表的精确记忆。

【选项原理深度拆解】

2. A. goto —错误（是关键字）

- 字节码铁证：

```
// goto 是 Java 保留关键字
// JLS §3.9 Keywords 明确列出
```

JVM 保障机制：虽然 Java 不使用 goto 跳转，但关键字表中保留它以防止冲突。

- 历史背景：源自 C/C++，Java 设计者认为 goto 破坏结构化编程。
- 命题陷阱破解：考生常误以为“不用就不是关键字”，实则“保留即关键字”。

3. B. const —错误（是关键字）

- 字节码铁证：

```
// const 也是保留关键字
// 尝试定义 const int x = 1; 会编译报错
```

JVM 保障机制：与 goto 一样，保留但禁用。Java 使用 `final` 代替常量。

- 易错点：C++ 程序员容易混淆，认为 const 可用。

4. C. null —正确（是字面量）

- **JLS** 规范依据：JLS §3.10.7 Null Literals。
- 本质区别：`null` 是表示空引用的字面量（Literal），不是关键字（Keyword）。

fifine专升本

- 字节码表现：

```
Object o = null;
// javap -c 输出:
// 0: aconst_null // 指令直接将 null 压栈，非关键字加载
// 1: astore_1
```

- 命题陷阱破解：极易混淆，因为 null 像关键字一样不能用作变量名。

5. **D. native** —错误（是关键字）
   - 功能：用于声明本地方法（JNI）。
   - 字节码标志：方法访问标志中包含 `ACC_NATIVE`。
   - 实战验证：

```
public native void method(); // 合法
```

【应背诵知识点（命题人思维版）】

(a) **Java** 保留但未使用的关键字：`goto`, `const`。

(b) 字面量非关键字：`null`, `true`, `false`。

(c) 选项辨析速查表：

选项	是否关键字	错误根源	记忆口诀
goto	是	unused 但保留	”goto 保留不用”
const	是	unused 但保留	”const 保留不用”
null	否	是字面量	”null 真值非关键字”
native	是	用于 JNI	”native 本地方法”

【命题趋势与实战策略】

- 考场秒杀：看到 `null`/`true`/`false` 选“非关键字”。
- 高频陷阱：混淆保留字与可用关键字。

fffine专升本

6. 关于自动装箱和拆箱的描述，正确的是？

A. 自动装箱是将基本类型转换为对应的包装类

B. 自动拆箱是将基本类型转换为包装类

C. 自动装箱发生在基本类型向包装类赋值时，需要显式调用方法

D. 自动拆箱不会发生 NullPointerException

答案与深度解析 答案：**A.** 自动装箱是将基本类型转换为对应的包装类

【核心认知框架】

- 题目本质：考查 Java 5 引入的 Autoboxing/Unboxing 机制及其底层实现。
- 关键区分点：装箱是基本 → 包装，拆箱是包装 → 基本；拆箱可能抛 NPE。

- 命题人意图：测试对编译器自动插入代码的理解及潜在风险。

【选项原理深度拆解】

7. **A.** 自动装箱是将基本类型转换为对应的包装类—正确
   - 字节码铁证：

```
Integer i = 5;
// javap -c 输出:
// 0: iconst_5
// 1: invokestatic #2 // Method java/lang/Integer.valueOf:(I)
    Ljava/lang/Integer;
```

   JVM 保障机制：编译器自动插入 `valueOf` 调用。
   - 缓存机制：-128 到 127 使用缓存池，超出范围 new 对象。

8. **B.** 自动拆箱是将基本类型转换为包装类—错误
   - 概念颠倒：拆箱是包装类 → 基本类型。
   - 字节码证据：

```
int x = i;
// javap -c 输出:
// 0: aload_1
// 1: invokevirtual #3 // Method java/lang/Integer.intValue:()I
```

9. **C.** 自动装箱发生在基本类型向包装类赋值时，需要显式调用方法—错误
   - 自动性：无需显式调用 `valueOf`，编译器自动完成。
   - 陷阱：显式调用是手动装箱，非自动。

10. **D.** 自动拆箱不会发生 **NullPointerException** —错误
    - 运行时风险：

```
Integer i = null;
int x = i; // 抛出 NullPointerException
```

    - 底层原因：拆箱调用 `intValue()`，对象为 null 时调用实例方法必抛 NPE。
    - 高频陷阱：集合中取 null 值直接赋给基本类型。

【应背诵知识点（命题人思维版）】

(a) 装箱方法：`Integer.valueOf()`, `Boolean.valueOf()` 等。

(b) 拆箱方法：`intValue()`, `booleanValue()` 等。

(c) **NPE** 陷阱：包装类为 null 时拆箱必挂。

【命题趋势与实战策略】

time专升本

- 考场秒杀：拆箱遇 null 必抛异常。
- 高阶延伸：== 比较包装类比较的是地址（除非缓存范围内）。

11. 下列哪个不是 Java 中合法的数组声明？

A. `int[] a = new int[5];`

B. `int a[] = {1,2,3};`

C. `int a[3] = {1,2,3};`

D. `int[] a = new int[]1,2,3;`

答案与深度解析 答案：**C.** `int a[3] = {1,2,3};`【核心认知框架】：考查数组声明语法规范，特别是声明时不能指定长度。【选项原理深度拆解】

12. **C.** 错误：声明数组变量时不能指定维度大小，大小在 new 时指定。

```
int a[3]; // 编译错误: ']' expected 或非法表达式
```

13. 其他选项：A 是标准声明，B 是 C 风格兼容声明，D 是匿名数组初始化，均合法。【应背诵知识点】：声明不指定大小，初始化才指定大小。

14. 在 Java 中，枚举类型的关键字是？

A. enum

B. Enum

C. enumeration

D. enumerator

答案与深度解析 答案：**A. enum** 【核心认知框架】：考查 Java 5 引入的枚举关键字，区分类名 Enum 和关键字 enum。【选项原理深度拆解】

15. **A.** 正确：小写 enum 是关键字。

16. **B.** 错误：Enum 是 `java.lang.Enum` 类名。【字节码证据】：ACC_ENUM 标志位。

17. 下列关于 Lambda 表达式的说法，错误的是？

A. Lambda 表达式可以替代匿名内部类的所有用法

B. Lambda 表达式只能用于函数式接口

fifine专升本

C. Lambda 表达式的参数类型可以省略

D. Lambda 表达式可以引用外部变量，但变量必须是 effectively final 的

答案与深度解析 答案：**A. Lambda** 表达式可以替代匿名内部类的所有用法【核心认知框架】：考查 Lambda 的局限性（this 指向、匿名类可重载方法等）。【选项原理深度拆解】

18. **A.** 错误：Lambda 不能替代需要重载方法或访问特定 this 的匿名内部类。

```
// 匿名类可以重载方法，Lambda 只能实现单一抽象方法
```

19. **D.** 正确：外部变量必须是 effectively final（JLS §4.12.4）。

20. 下列哪个接口是函数式接口（只有一个抽象方法）？

A. Runnable

B. Comparator

C. Serializable

D. Cloneable

答案与深度解析 答案：**A. Runnable**【核心认知框架】：考查函数式接口定义（@FunctionalInterface）。【选项原理深度拆解】

21. **A.** 正确：只有 `run()` 一个抽象方法。

22. **C/D.** 错误：标记接口，无抽象方法。

23. **B.** 错误：`Comparator` 有 `compare` 和 `equals` (继承自 Object 不算)，但通常视为函数式，但 Runnable 更典型。注：Comparator 也是函数式接口，但题目单选通常选最典型。若严格看，Comparator 也是。此处题目设计 Runnable 为最优解。

24. 关于 Stream API 的描述，正确的是？

A. Stream 可以重复使用

B. Stream 的中间操作会立即执行

C. Stream 的终端操作会触发流的执行

D. Stream 只能用于集合，不能用于数组

fifine专升本

答案与深度解析 答案：**C. Stream** 的终端操作会触发流的执行 【核心认知框架】：考查 Stream 的懒加载特性及一次性消费原则。【选项原理深度拆解】

25. **A.** 错误：Stream 是一次性的，使用后关闭。

26. **B.** 错误：中间操作是懒加载（Lazy）。

27. **D.** 错误：`Arrays.stream()` 支持数组。

28. 下列哪个类用于表示不可变日期（Java 8 及以上）?

A. Date

B. Calendar

C. LocalDate

D. SimpleDateFormat

答案与深度解析 答案：**C. LocalDate** 【核心认知框架】：考查 Java 8 Date/Time API 的不可变性设计。【选项原理深度拆解】

29. **C.** 正确：`java.time` 包下的类均为不可变且线程安全。

30. **A/B/D.** 错误：旧 API 可变且线程不安全。

31. 关于 Optional 类的说法，正确的是？

A. Optional 可以包含 null 值

B. Optional.of() 方法允许传入 null

C. Optional.ofNullable() 允许传入 null

D. Optional 主要用于提高性能

答案与深度解析 答案：**C. Optional.ofNullable()** 允许传入 **null** 【核心认知框架】：考查 Optional 的构造方法及设计意图（避免 NPE）。【选项原理深度拆解】

32. **A.** 错误：Optional 本身不能为 null，内部值可以为 null 但需用 ofNullable。

33. **B.** 错误：`of(null)` 抛 NPE。

34. **C.** 正确：`ofNullable` 接受 null 返回 empty。

fitime专升本

35. 在泛型中，`? extends Number` 表示？

A. 类型参数必须是 Number 或 Number 的子类

B. 类型参数必须是 Number 或 Number 的父类

C. 类型参数必须是 Number

D. 类型参数可以是任意类型

答案与深度解析 答案：**A.** 类型参数必须是 **Number** 或 **Number** 的子类 【核心认知框架】：考查泛型通配符 PECS 原则（Producer Extends）。【选项原理深度拆解】

36. A. 正确：上界通配符，限制为子类。

37. B. 错误：`? super Number` 才是父类。

38. 关于 Java 注解（Annotation）的描述，正确的是？

A. 注解可以影响程序的运行逻辑

B. 注解只能用于类和方法

C. `@Override` 注解用于标识重写父类方法

D. 自定义注解需要使用 `@interface` 关键字

答案与深度解析 答案：**D.** 自定义注解需要使用 `@interface` 关键字 【核心认知框架】：考查注解的定义及使用范围。【选项原理深度拆解】

39. A. 错误：注解本身不影响逻辑，需配合反射或编译器处理。

40. C. 错误：`@Override` 是编译期检查，非运行时逻辑。

41. D. 正确：定义注解用 `@interface`。

42. 下列哪个反射方法可以获取类的所有 public 字段？

A. getDeclaredFields()

B. getFields()

C. getDeclaredField(String name)

D. getField(String name)

fifine专升本

答案与深度解析 答案：**B. getFields()** 【核心认知框架】：考查反射 API 中 `getFields` 与 `getDeclaredFields` 的区别。【选项原理深度拆解】

43. **A.** 错误：`getDeclaredFields` 获取所有字段（含 private），不含继承。

44. **B.** 正确：`getFields` 获取所有 public 字段（含继承）。

45. 关于并发包中的 ConcurrentHashMap，正确的是？

A. 它是线程安全的，且不允许 null 键和值

B. 它的所有方法都使用 synchronized 修饰

C. 它继承自 HashMap

D. 它不支持高并发

答案与深度解析 答案：**A.** 它是线程安全的，且不允许 **null** 键和值 【核心认知框架】：考查 ConcurrentHashMap 的设计特性（CAS+synchronized）。【选项原理深度拆解】

46. **A.** 正确：JDK8+ 禁止 null 以防歧义。

47. **B.** 错误：使用 CAS 和节点锁，非全表 synchronized。

48. **C.** 错误：继承自 `AbstractMap`。

49. 下列关于 volatile 关键字的说法，正确的是？

A. volatile 可以保证原子性

B. volatile 可以保证可见性

C. volatile 可以替代 synchronized

D. volatile 修饰的变量不会被线程缓存

答案与深度解析 答案：**B. volatile** 可以保证可见性【核心认知框架】：考查 JMM 中 volatile 的语义（可见性、有序性，非原子性）。【选项原理深度拆解】

50. **A.** 错误：`i++` 非原子。

51. **B.** 正确：强制从主内存读取。

fifine专升本

52. **D.** 错误：会被缓存，但强制刷新。

53. 在 Java NIO 中，Buffer 的 flip() 方法的作用是？

A. 清空缓冲区

B. 反转缓冲区，准备读取操作

C. 标记当前位置

D. 重置位置到标记

答案与深度解析 答案：**B.** 反转缓冲区，准备读取操作 【核心认知框架】：考查 NIO Buffer 模式切换（Write→Read）。【选项原理深度拆解】

54. **B.** 正确：`limit=position, position=0`。

55. **A.** 错误：`clear()` 是清空。

56. 关于 try-with-resources 语句，正确的是？

A. 可以自动关闭实现了 AutoCloseable 接口的资源

B. 只能关闭一个资源

C. 不需要 finally 块，但需要 catch 块

D. 资源变量必须是 final 的

答案与深度解析 答案：**A.** 可以自动关闭实现了 **AutoCloseable** 接口的资源【核心认知框架】：考查 Java 7 引入的自动资源管理。【选项原理深度拆解】

57. **A.** 正确：编译器自动插入 `close()` 调用。

58. **B.** 错误：支持多个资源。

59. 下列哪个类不是 java.time 包中的？

A. LocalDateTime

B. ZonedDateTime

C. DateTimeFormatter

D. DateFormat

ffine专升本

答案与深度解析 答案：**D. DateFormat** 【核心认知框架】：考查新旧日期 API 包的区分。【选项原理深度拆解】

60. **D.** 正确：`DateFormat` 属于 `java.text` 包（旧 API）。

61. 关于 Java 正则表达式的说法，正确的是？

A. Pattern 类用于匹配字符串

B. Matcher 类的 matches() 方法要求整个字符串匹配

C. find() 方法只匹配一次

D. group() 方法返回整个匹配结果

答案与深度解析 答案：**B. Matcher 类的 matches() 方法要求整个字符串匹配** 【核心认知框架】：考查正则 API 的使用细节。【选项原理深度拆解】

62. **A.** 错误：`Pattern` 是编译，`Matcher` 是匹配。

63. **B.** 正确：`matches` 全匹配，`find` 子匹配。

64. 在 JDBC 中，使用哪个接口调用存储过程？

A. Statement

B. PreparedStatement

C. CallableStatement

D. ProcedureStatement

答案与深度解析 答案：**C. CallableStatement** 【核心认知框架】：考查 JDBC 接口层级及用途。【选项原理深度拆解】

65. **C.** 正确：`CallableStatement` 专用于存储过程。

66. 关于 Swing 中 JTable 的说法，正确的是？

A. JTable 默认使用 TableModel 存储数据

B. JTable 的列宽不能调整

C. JTable 不能添加滚动条

D. JTable 中的数据不能编辑

fifine专升本

答案与深度解析 答案：**A. JTable** 默认使用 **TableModel** 存储数据 【核心认知框架】：考查 Swing 组件 MVC 模式。【选项原理深度拆解】

67. **A.** 正确：数据与视图分离。

68. **B/C/D.** 错误：均可调整/添加/编辑。

2 判断题（共 10 小题，每小题 2 分，共 20 分）

（正确的选 *A*，错误的选 *B*）

1. 抽象类可以包含构造方法。（ A/B ）

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查抽象类构造器的作用（供子类调用）。【选项原理深度拆解】

2. **A.** 正确：抽象类构造器用于初始化父类部分，子类构造器第一行隐式调用 `super()`。

```
abstract class A { A() { } } // 合法
```

3. **B.** 错误：若选 B 则误解抽象类不能实例化即无构造器。【应背诵知识点】：抽象类不能 new，但有构造器。

4. Java 中所有类都直接或间接继承自 Object 类。（ A/B ）

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查 Java 类层次结构的根。【选项原理深度拆解】

5. **A.** 正确：JLS 规定，无显式父类则默认继承 Object。

6. 字节码证据：`javap` 显示 `extends java/lang/Object`。

7. 接口中的 default 方法可以有方法体。（ A/B ）

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查 Java 8 接口演进。【选项原理深度拆解】

8. **A.** 正确：`default` 方法允许有实现，解决接口升级问题。

9. 使用 ‘==‘ 比较两个字符串时，比较的是字符串的内容。（ A/B ）

tfine专升本

答案与深度解析 答案：**B** (错误) 【核心认知框架】：考查引用比较 vs 内容比较。【选项原理深度拆解】

10. **B.** 正确：`==` 比较地址，`equals()` 比较内容。

```
new String("a") == new String("a") // false
```

11. 一个 try 块可以对应多个 catch 块，且父类异常的 catch 块必须放在子类之后。（ A/B ）

答案与深度解析 答案：**A** (正确) 【核心认知框架】：考查异常捕获的多态性及顺序规则。【选项原理深度拆解】

12. **A.** 正确：子类异常更具体，必须先捕获，否则父类捕获后子类 unreachable。

13. ArrayList 的初始容量是 10。（ A/B ）

答案与深度解析 答案：**A** (正确) 【核心认知框架】：考查集合默认容量常量。【选项原理深度拆解】

14. **A.** 正确：JDK 源码中 `DEFAULT_CAPACITY = 10`。

15. 泛型通配符 ‘¿ 可以用于创建对象，例如 ‘new ArrayList<>()‘。（ A/B ）

答案与深度解析 答案：**B** (错误) 【核心认知框架】：考查泛型擦除及实例化限制。【选项原理深度拆解】

16. **B.** 正确：`new ArrayList<?>()` 编译错误，泛型实例化必须是具体类型。

```
new ArrayList<?>(); // 编译错误
```

17. 线程调用 wait() 方法后，会释放持有的对象锁。（ A/B ）

答案与深度解析 答案：**A** (正确) 【核心认知框架】：考查 wait/notify 机制及锁释放行为。【选项原理深度拆解】

18. **A.** 正确：`wait()` 原子操作：释放锁 + 进入等待集。

19. FileInputStream 和 FileOutputStream 是字节流。（ A/B ）

fine专升本

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查 IO 流分类（字节 vs 字符）。【选项原理深度拆解】

20. **A.** 正确：`InputStream/OutputStream` 结尾均为字节流。

21. 在 UDP 通信中，DatagramPacket 用于封装数据报。（ A/B ）

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查网络编程 UDP 包结构。【选项原理深度拆解】

22. **A.** 正确：`DatagramPacket` 承载数据，`DatagramSocket` 发送。

3 填空题（共 5 小题，每空 2 分，共 20 分）

1. Java 中，声明包使用 _____ 关键字，声明接口使用 _____ 关键字。

答案与深度解析 答案：**package, interface** 【核心认知框架】：考查基础语法关键字。【选项原理深度拆解】

2. 空 1：`package` 定义命名空间，必须位于文件第一行（注释除外）。

3. 空 2：`interface` 定义接口类型，隐含 `public abstract` 方法。【字节码证据】：`package` 对应 class 文件中的 `this_package` 索引。

4. 面向对象程序设计的基本特征包括封装、继承和 _____。

答案与深度解析 答案：多态 【核心认知框架】：考查 OOP 三大/四大特征（通常填多态，有时含抽象）。【选项原理深度拆解】

5. 多态：同一操作作用于不同对象产生不同行为（重载/重写）。

6. 异常分为两类：受检查异常（checked exception）和 _____ 异常。

答案与深度解析 答案：非受检查（**unchecked**）/ 运行时 【核心认知框架】：考查异常体系分类。【选项原理深度拆解】

7. **Unchecked**：继承自 `RuntimeException`，编译器不强制 catch。

8. 集合框架中，Set 接口的特点是元素 _____ 且 _____。

ffine专升本

答案与深度解析 答案：不可重复，无序（或唯一）【核心认知框架】：考查 Set 集合特性。【选项原理深度拆解】

9. 不可重复：add 重复元素返回 false。

10. 无序：HashSet 不保证顺序（LinkedHashSet 除外）。

11. Java 8 中，引入的函数式编程特性包括 Lambda 表达式和 _____API。

答案与深度解析 答案：**Stream** 【核心认知框架】：考查 Java 8 新特性。【选项原理深度拆解】

12. **Stream**：用于集合数据的函数式操作。

4 程序运行结果题（共 5 小题，每小题 6 分，共 30 分）

1.

```java
public class TestA {
public static void main(String[] args) {
int x = 5;
int y = ++x + x++ + x;
System.out.println(x);
System.out.println(y);
}
}
```

答案与深度解析 输出：**7, 19** 【核心认知框架】：考查自增运算符的前缀/后缀优先级及求值顺序。【执行流程深度拆解】

- 初始：x=5。
- **++x**：先加后用，x=6，表达式值 6。
- **x++**：先用后加，表达式值 6，x 变为 7。
- **+ x**：此时 x=7，表达式值 7。
- 计算 **y**：6 + 6 + 7 = 19。
- 最终 **x**：7。

【命题陷阱破解】：切勿混淆运算顺序，Java 从左到右求值。

2.

```java
class Animal {
public void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
```

fifine专升本

```
public void sound() { System.out.println("Bark"); }
}
public class TestB {
public static void main(String[] args) {
Animal a = new Dog();
a.sound();
}
}
```

答案与深度解析 输出：**Bark**【核心认知框架】：考查多态与方法动态绑定。【执行流程深度拆解】

- 编译时类型：Animal。
- 运行时类型：Dog。
- 动态绑定：JVM 根据堆中实际对象类型调用 Dog.sound()。

【字节码证据】：invokevirtual 指令支持动态分派。

3.

```
public class TestC {
public static void main(String[] args) {
String s = "Java";
change(s);
System.out.println(s);
}
public static void change(String str) {
str = "Python";
}
}
```

答案与深度解析 输出：**Java**【核心认知框架】：考查 String 不可变性及参数传递机制。【执行流程深度拆解】

- 传递机制：引用传递副本。
- 方法内：str 指向新对象"Python"，不影响外部 s。
- **String** 不可变：无法修改原对象内容。

4.

```
import java.util.*;
public class TestD {
public static void main(String[] args) {
List<String> list = Arrays.asList("A", "B", "C");
list.stream()
```

fifine专升本

```
.filter(s -> !s.equals("B"))
.map(s -> s.toLowerCase())
.forEach(System.out::print);
}
}
```

答案与深度解析 输出：**ac** 【核心认知框架】：考查 Stream 中间操作链式调用。【执行流程深度拆解】

- **Filter**：过滤掉"B"，剩余"A", "C"。
- **Map**：转为小写"a", "c"。
- **ForEach**：打印，无分隔符。

5.

```
public class TestE {
static int count = 0;
public static void main(String[] args) {
try { method(); }
catch (StackOverflowError e) { System.out.println("Overflow"); }
System.out.println(count);
}
public static void method() {
count++;
method();
}
}
```

答案与深度解析 输出：**Overflow, (一个大数字)** 【核心认知框架】：考查递归深度及 StackOverflowError 捕获。【执行流程深度拆解】

- 递归：`method` 无限调用，栈帧耗尽。
- **Count**：每次调用前 `count++`，直到栈满。
- **Catch**：`Error` 可被 catch 捕获（不推荐但合法）。
- 结果：打印 Overflow 和递归次数。

fifine专升本

5 编程题（共 2 小题，每小题 20 分，共 40 分）

1. 设计一个简单的在线购物系统

答案与深度解析【核心认知框架】: 考查接口实现、多态、集合使用及封装。【设计模式深度拆解】

- 接口设计：Product 定义标准，实现解耦。
- 多态存储：ArrayList<Product> 存储不同子类。
- 封装：属性私有，通过方法访问。

【代码实现关键点】

```java
interface Product { String getName(); double getPrice(); }
class ShoppingCart {
    private List<Product> items = new ArrayList<>();
    public void addProduct(Product p) { items.add(p); }
    public double calculateTotal() {
        return items.stream().mapToDouble(Product::getPrice).sum();
    }
}
```

【命题陷阱破解】: 注意总价计算精度（实际应用应用 BigDecimal，考试 double 即可）。

2. 实现一个图书管理类

答案与深度解析【核心认知框架】: 考查 HashSet 唯一性原理（equals/hashCode）、集合操作。【设计模式深度拆解】

- 唯一性：HashSet 依赖 hashCode 和 equals。
- **ISBN** 标识：重写方法仅比较 ISBN。
- 查找：流式 API 或遍历。

【代码实现关键点】

```java
class Book {
    // 必须重写 equals 和 hashCode
    public boolean equals(Object o) {
        return o instanceof Book && ((Book)o).isbn.equals(this.isbn
            );
    }
    public int hashCode() { return isbn.hashCode(); }
}
```

【命题陷阱破解】: 忘记重写 hashCode 会导致 HashSet 无法去重。

——讲义结束，祝学习顺利——

time专升本